Ứng dụng hệ đa tác tử dựa trên mô hình ngôn ngữ nhỏ trong thiết kế giảng dạy

Bùi Việt Phương[1], Bùi Văn Tâm[2], Phạm Văn Hải[1,*]
[1]Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội
[2]Khoa Công nghệ Thông tin, Trường Đại học Hải Phòng
Email: phuong.bv242011m@sis.hust.edu.vn, tambv@dhhp.edu.vn, haipv@soict.hust.edu.vn
[*]Tác giả liên hệ

***Tóm tắt*—Các hệ thống tác tử dựa trên mô hình ngôn ngữ lớn (Large Language Model – LLM) phục vụ thiết kế giảng dạy hiện nay chủ yếu phụ thuộc vào các mô hình thương mại được truy cập thông qua API, trong khi nhiều cơ sở giáo dục có nhu cầu sử dụng các mô hình ngôn ngữ nhỏ (Small Language Model – SLM) mã nguồn mở có khả năng vận hành tại chỗ. Nghiên cứu này trình bày một hệ thống đa tác tử hỗ trợ thiết kế giảng dạy trên SLM, gồm sáu tác tử chuyên trách tương ứng với các công đoạn của quá trình thiết kế. Nhằm duy trì tính nhất quán giữa mục tiêu học tập, hoạt động dạy–học và đánh giá, các sản phẩm thiết kế được liên kết thông qua một đồ thị căn chỉnh dùng chung. Trong quá trình xử lý, hệ thống kiểm tra các sản phẩm do từng tác tử tạo ra, ghi nhận các quan hệ trên đồ thị và lưu các vi phạm để hỗ trợ giảng viên truy vết, rà soát bản thiết kế. Trên cơ sở kiến trúc đề xuất, một ứng dụng web được xây dựng và minh họa thông qua nghiên cứu tình huống đối với học phần Nhập môn Học máy. Kết quả nghiên cứu tình huống cho thấy ứng dụng có khả năng tạo các thành phần của bản thiết kế, bao gồm mục tiêu học tập, mô-đun, nhiệm vụ đánh giá, hoạt động, tài liệu và kế hoạch buổi học; sổ vi phạm ghi nhận 8 vi phạm, trong đó 2 vi phạm được xử lý trong quá trình sinh và 6 vi phạm được lưu lại để giảng viên xem xét. Bên cạnh đó, thực nghiệm trên 30 kịch bản của ISD-Agent-Bench với Qwen3.5-2B cho thấy hệ thống đề xuất đạt điểm ADDIE 59.0; không ghi nhận khác biệt có ý nghĩa thống kê so với React-ADDIE, với điểm ADDIE 59.1. Kết quả cho thấy chất lượng ADDIE của kiến trúc đề xuất cạnh tranh với các tác tử đối chứng trên cùng SLM, đồng thời cung cấp khả năng truy vết cấu trúc nhằm hỗ trợ giảng viên phát hiện và rà soát các vấn đề trong bản thiết kế. Tuy nhiên, sản phẩm do SLM tạo ra vẫn cần được giảng viên thẩm định trước khi đưa vào sử dụng.**

***Từ khóa*—hệ đa tác tử (multi-agent system); mô hình ngôn ngữ nhỏ (small language model); thiết kế giảng dạy (instructional design); constructive alignment**

I. GIỚI THIỆU

Thiết kế giảng dạy (Instructional Systems Design – ISD) là quá trình xây dựng một khóa học, từ phân tích nhu cầu đến kế hoạch triển khai. Mô hình ADDIE tổ chức quá trình này thành năm pha: phân tích, thiết kế, phát triển, triển khai và đánh giá [1]. Với giảng viên, mỗi pha đòi hỏi nhiều công sức: viết mục tiêu, ra đề, thiết kế hoạt động, soạn tài liệu và lập kế hoạch từng buổi. Các hệ tác tử dựa trên mô hình ngôn ngữ lớn (Large Language Model – LLM) đã có thể tự động hóa nhiều phần của quy trình này, từ kế hoạch bài giảng đến trọn bộ học liệu của một khóa học [2]–[4]. Bộ đánh giá chuẩn (benchmark) ISD-Agent-Bench so sánh các tác tử ISD trên một tập kịch bản chung [5].

Tuy nhiên, các hệ tác tử ISD nói trên chủ yếu được xây dựng và kiểm chứng trên LLM cỡ lớn, phần lớn là LLM thương mại gọi qua API. Chi phí và nguồn lực tài chính là rào cản lớn khi các trường đại học ứng dụng trí tuệ nhân tạo [6]. Khi gọi API, dữ liệu học phần và người học còn phải đi qua hạ tầng của bên thứ ba, trong khi việc dùng LLM trong giáo dục đã gây lo ngại về quyền riêng tư dữ liệu người học [7]. Vì vậy, các cơ sở giáo dục quan tâm đến mô hình ngôn ngữ nhỏ (Small Language Model – SLM) mã nguồn mở, vận hành tại chỗ [8], [9].

Mặt khác, SLM kém hơn LLM ở các tác vụ nhiều bước dùng công cụ [10], trong khi một bản thiết kế khóa học dài và gồm nhiều phần phụ thuộc nhau. Yêu cầu cốt lõi giữa các phần này là constructive alignment [11]: mục tiêu học tập nêu người học phải làm được gì, hoạt động dạy–học cho người học luyện việc đó, và đánh giá đo đúng việc đó. Ví dụ, khi mục tiêu yêu cầu sinh viên đánh giá một mô hình học máy, câu hỏi kiểm tra không thể chỉ hỏi lại định nghĩa. Trong các hệ tác tử ISD nêu trên, quan hệ này chỉ nằm ngầm trong văn bản. Các hệ này không kiểm tra quan hệ đó, nên khi mô hình nhỏ đi, lỗi lệch có thể dễ xảy ra hơn và đến thẳng giảng viên.

Bài báo có hai mục tiêu. Thứ nhất, xây dựng một hệ thống chạy trên SLM tự triển khai để sinh bản thiết kế giảng dạy cho một học phần, gồm mục tiêu học tập, mô-đun, đánh giá, hoạt động, tài liệu và kế hoạch buổi học, với chất lượng cạnh tranh với các hệ tác tử hiện có khi dùng cùng một SLM. Thứ hai, giúp giảng viên phát hiện và truy vết những chỗ mục tiêu, hoạt động và đánh giá chưa khớp nhau mà không phải đọc lại toàn bộ bản thiết kế.

Đóng góp kỹ thuật của bài báo gồm ba điểm. (i) Một kiến trúc đa tác tử (Multi-Agent System – MAS) gồm sáu tác tử theo công đoạn, cùng làm việc trên một đồ thị căn chỉnh tường minh; mỗi quan hệ mục tiêu–hoạt động–đánh giá là một cạnh có kiểu, do lớp điều phối của hệ thống (orchestration layer) ghi từ id mà tác tử khai báo. (ii) Một cơ chế kiểm tra và sửa trong quá trình sinh: mỗi lần nộp được kiểm tra theo định dạng và theo quan hệ trên đồ thị; vi phạm cấu trúc được chuẩn hóa tự động, vi phạm cần viết nội dung được trả lại cho tác tử phụ trách, và mọi vi phạm được ghi vào sổ vi phạm cùng kết quả xử lý. (iii) Một ứng dụng web hiển thị đồ thị và sổ vi

phạm để giảng viên truy vết bản thiết kế. Chúng tôi đánh giá hệ thống trên ISD-Agent-Bench với Qwen3.5-2B và qua một nghiên cứu tình huống trên học phần Nhập môn Học máy với đầu ra tiếng Việt.

II. CÁC NGHIÊN CỨU LIÊN QUAN

A. Hệ tác tử cho thiết kế giảng dạy

COMA dùng ba tác tử theo vai phối hợp và lặp lại để tinh chỉnh kế hoạch bài giảng [2]. EduPlanner dùng nhiều tác tử LLM đánh giá và chỉnh sửa thiết kế bài giảng [3]. Instructional Agents sinh trọn gói học liệu của một khóa học bằng các tác tử theo vai [4]. ISD-Agent-Bench tập hợp các tác tử theo nhiều lý thuyết ISD, trong đó có một tác tử theo mẫu ReAct [5], [12]. Về kiến trúc, COMA, EduPlanner và Instructional Agents cùng nhóm MAS theo quy trình vận hành chuẩn như MetaGPT, trong đó các vai và thứ tự của chúng được định trước [13], [14]. Các hệ này chủ yếu được xây dựng và kiểm chứng trên mô hình lớn và giữ sự nhất quán giữa các thành phần một cách ngầm định.

B. SLM trong giáo dục và constructive alignment

Chan và cộng sự so sánh các cỡ mô hình vận hành tại chỗ cho trường học [8]. Katharakis và cộng sự cho thấy SLM có 7–17 tỉ tham số chạy trên một máy chủ GPU có thể làm trợ giảng theo chương trình học [9]. Các nghiên cứu này không nhắm tới trọn quy trình thiết kế giảng dạy. Pereira và cộng sự dùng ChatGPT để thiết kế khóa học theo constructive alignment và nhận thấy kết quả vẫn cần giảng viên hoàn thiện [15]. Thang phân loại Bloom sửa đổi cung cấp bộ từ vựng chung về mức nhận thức [16]. Hệ thống đề xuất dùng thang này để so mức nhận thức mà câu hỏi và hoạt động khai báo với mục tiêu.

III. MÔ HÌNH HỆ THỐNG ĐỀ XUẤT

A. Kiến trúc tổng quát

Hình 1 trình bày kiến trúc tổng quát của hệ thống. Đầu vào là một bản mô tả yêu cầu thiết kế (design brief): mục tiêu khóa học, hồ sơ người học, bối cảnh và ràng buộc. Kiến trúc đề xuất gồm ba lớp. Lớp thiết kế và lập kế hoạch gồm sáu tác tử, mỗi tác tử đảm nhận một công đoạn. Lớp hỗ trợ SLM là lớp điều phối bao quanh mô hình. Lớp này giữ đồ thị căn chỉnh, kiểm tra từng sản phẩm, trả lỗi để tác tử sửa và ghi sổ vi phạm. Lớp dạy và học là nơi giảng viên xem, truy vết và duyệt bản thiết kế. Mô hình viết nội dung giảng dạy và quyết định cách thực hiện từng công đoạn. Lớp điều phối quyết định thứ tự công đoạn, định dạng của từng sản phẩm, việc ghi đồ thị và việc kiểm tra.

Đầu ra gồm bản thiết kế khóa học theo cấu trúc ADDIE, đồ thị căn chỉnh và sổ vi phạm. Bản thiết kế có mục tiêu và mô-đun, kế hoạch đánh giá kèm câu hỏi và rubric, hoạt động, tài liệu, kế hoạch buổi học và tài liệu giảng viên; sổ vi phạm (violation ledger) ghi các vi phạm cùng kết quả xử lý.

Bảng I
SÁU CÔNG ĐOẠN, SẢN PHẨM VÀ CÁC KIỂM TRA CHÍNH

Công đoạn	Sản phẩm	Kiểm tra chính
1. Phân tích nhu cầu	khoảng trống năng lực, hồ sơ người học, chủ đề, ràng buộc	đủ các trường bắt buộc
2. Mục tiêu, mô-đun	mục tiêu kèm động từ và mức Bloom; mô-đun kèm thời lượng	động từ khớp mức Bloom; mô-đun phủ mục tiêu
3. Đánh giá	bảng đặc tả, nhiệm vụ đánh giá, câu hỏi kèm rubric	mỗi mục tiêu được đánh giá, có câu hỏi ở mức không thấp hơn mục tiêu
4. Hoạt động	hoạt động học tập theo mô-đun	hoạt động luyện mục tiêu và chuẩn bị cho đánh giá; ngân sách thời gian
5. Tài liệu	lựa chọn phương tiện, tài liệu	tài liệu thuộc mô-đun, đúng phương tiện đã chọn
6. Triển khai	kế hoạch buổi học, tài liệu giảng viên, tài liệu vận hành	mọi hoạt động và tài liệu được xếp vào buổi học

B. Sáu công đoạn

Bảng I liệt kê sáu công đoạn. Thứ tự công đoạn tuân theo Backward Design: xác định kết quả mong muốn trước, sau đó đến bằng chứng đánh giá và cuối cùng là kế hoạch học tập [17]. Thứ tự này cố định. Tác tử tự chủ trong cách thực hiện một công đoạn nhưng không chọn công đoạn kế tiếp. Ở bốn công đoạn sau, số sản phẩm tăng theo quy mô khóa học. Các sản phẩm này do tầng tác tử phát triển (Developer) viết theo từng mô-đun. Mỗi Developer nhận một nhóm việc được suy ra từ thiết kế đã có. Bộ tổng hợp sắp xếp nội dung thành năm pha ADDIE theo định dạng mà benchmark yêu cầu.

C. Đồ thị căn chỉnh

Trạng thái dùng chung của các tác tử là một đồ thị có hướng

$$G = (V, E), \qquad E \subseteq V \times V \times \mathcal{R}. \tag{1}$$

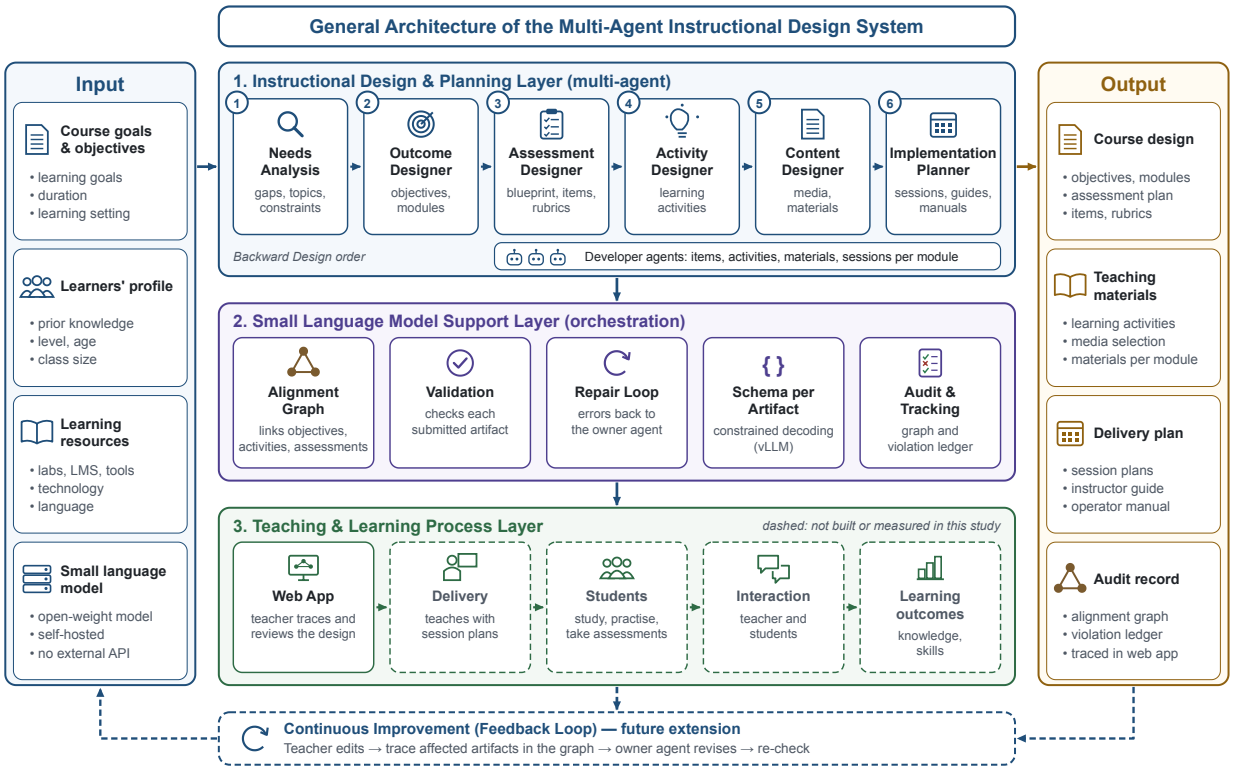
Mỗi nút $v \in V$ là một sản phẩm thiết kế có kiểu, như mục tiêu, mô-đun, nhiệm vụ đánh giá, hoạt động hay tài liệu. Mỗi cạnh (u, v, r) nối hai sản phẩm qua một kiểu quan hệ $r \in \mathcal{R}$. Ký hiệu $E_r = \{(u, v) \colon (u, v, r) \in E\}$. Ba quan hệ cốt lõi là ba cạnh của tam giác constructive alignment: C_2 nối mục tiêu với nhiệm vụ đánh giá đo nó, C_3 nối mục tiêu với hoạt động luyện tập nó, và C_4 nối hoạt động với nhiệm vụ đánh giá mà nó chuẩn bị. Các quan hệ còn lại ghi nguồn gốc và cách tổ chức theo mô-đun.

Một mục tiêu o được căn chỉnh về cấu trúc khi tam giác khép kín trên nó:

$$\begin{aligned}\text{aligned}(o) \iff \exists\, h, d \colon\ & (o, h) \in E_{C_3} \wedge (o, d) \in E_{C_2} \\ & \wedge\, (h, d) \in E_{C_4},\end{aligned} \tag{2}$$

trong đó h là một hoạt động và d là một nhiệm vụ đánh giá. Ba cạnh phải gặp nhau trên cùng một hoạt động và cùng một nhiệm vụ. Với mỗi quan hệ r, gọi S_r là tập nút được xét (mục tiêu với C_2 và C_3, nhiệm vụ đánh giá với C_4) và $N_r(v)$ là tập nút nối với v qua r. Độ phủ của r là

$$\text{cov}(r) = \frac{\left|\{v \in S_r \colon N_r(v) \neq \emptyset\}\right|}{|S_r|}. \tag{3}$$



Hình 1. Kiến trúc tổng quát của hệ thống. Lớp 1 gồm sáu tác tử theo công đoạn; lớp 2 là lớp điều phối quanh SLM; lớp 3 là quá trình dạy và học, nơi giảng viên dùng bản thiết kế. Các khối nét đứt ở lớp 3 và vòng cải tiến là bối cảnh sử dụng và hướng mở rộng, không thuộc phạm vi thực nghiệm.

Ví dụ, $\mathrm{cov}(C_2)$ là tỉ lệ mục tiêu có ít nhất một nhiệm vụ đánh giá đo nó. Hai đại lượng này cho biết quan hệ có tồn tại hay không, không cho biết thiết kế có giá trị sư phạm hay không. Một cạnh vẫn được tính khi phép kiểm tra mức Bloom gắn cờ nó; cờ này được báo riêng trong sổ vi phạm.

Mô hình không trực tiếp tạo cạnh. Với ba quan hệ cốt lõi, mỗi sản phẩm khai báo mã định danh (id) của sản phẩm mà nó phục vụ, ví dụ một hoạt động khai báo mục tiêu mà nó luyện tập. Khi ghi sản phẩm, lớp điều phối tra từng id và tự ghi cạnh có kiểu. Lần nộp mà không id nào trong một trường tham chiếu trỏ tới nút có thật bị từ chối. Vì vậy, độ phủ đo việc mỗi sản phẩm được gắn tới một đích hợp lệ, không đo sự phù hợp về nội dung. Mức Bloom của mục tiêu, câu hỏi và hoạt động cũng do tác tử khai báo; các phép kiểm tra so sánh những nhãn và id này, không đọc lại nội dung văn bản.

D. Kiểm tra, sửa và khả năng mở rộng

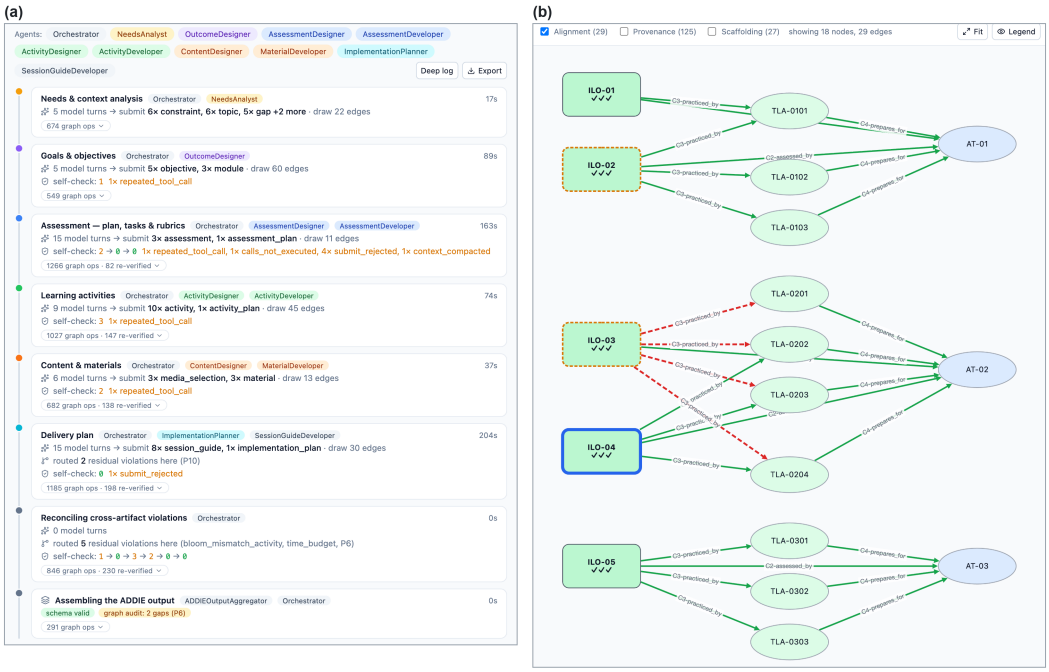
Mỗi tác tử làm việc theo vòng lặp hành động–quan sát theo kiểu ReAct [12], hiện thực bằng cơ chế gọi công cụ của mô hình. Tác tử đọc phần đồ thị thuộc phạm vi vai của mình, rồi nộp sản phẩm qua một công cụ nộp riêng. Lớp điều phối kiểm tra mỗi lần nộp theo định dạng của sản phẩm. Lần nộp hợp lệ được ghi thành nút kèm các cạnh. Lần nộp không hợp lệ bị từ chối, và thông báo lỗi trở thành quan sát kế tiếp để tác tử sửa. Sau mỗi công đoạn, các phép kiểm tra trong Bảng I chạy mà không gọi mô hình. Lớp điều phối không tự sinh mới các thành phần nội dung giảng dạy; các sửa tự động chỉ giới hạn ở các phép chuẩn hóa cấu trúc hoặc từ vựng được định nghĩa trước. Các vi phạm cần viết thêm nội dung được trả lại cho tác tử phụ trách khi tác tử đó còn đang làm việc; vi phạm phát hiện muộn hơn được ghi vào sổ để giảng viên xem. Riêng bản tóm tắt kiểm tra và một số đoạn mô tả tổng hợp trong đầu ra do lớp điều phối sinh từ sổ vi phạm và từ mẫu cố định.

Vì mỗi sản phẩm là một nút có id và mỗi quan hệ là một cạnh có kiểu, kiến trúc tạo nền tảng cho hướng mở rộng sang vòng cải tiến cục bộ (khối nét đứt ở Hình 1): khi giảng viên sửa một sản phẩm, hệ thống lần theo cạnh để tìm các sản phẩm liên quan, giao lại cho đúng tác tử và chỉ kiểm tra lại phần đó. Vòng cải tiến này chưa được hiện thực trong nghiên cứu này.

IV. THỰC NGHIỆM VÀ ĐÁNH GIÁ

A. Thiết lập thực nghiệm

Chúng tôi dùng kịch bản, sáu tác tử đối chứng, rubric và bộ chấm của ISD-Agent-Bench [5], với bộ chạy thực nghiệm của chúng tôi. Tập đánh giá gồm 30 kịch bản tiếng Anh thuộc tập kiểm tra, tách biệt với 10 kịch bản đã dùng khi hoàn thiện hệ thống. Sáu tác tử đối chứng là EduPlanner (bản hiện thực lại trong benchmark [3]), Baseline (sinh một lượt), React-ADDIE (kết hợp ADDIE với ReAct [12]; prompt được dịch sang tiếng Anh), ADDIE-Agent, Dick-Carey-Agent và RPISD-Agent. Mọi hệ chạy trên cùng Qwen3.5-2B, phục vụ bằng vLLM [18] trên máy chủ GPU tự triển khai, với temperature 0.7 (EduPlanner giữ temperature riêng cho từng vai), trần 8192 token mỗi lượt gọi và chế độ suy luận giới hạn ở 32 token, cố



Hình 2. Ứng dụng web trên học phần Nhập môn Học máy. (a) Quy trình: sáu công đoạn, bước đối chiếu vi phạm và bước tổng hợp, kèm tác tử, sản phẩm đã nộp và kết quả tự kiểm tra. (b) Đồ thị căn chỉnh của khóa học ở lớp quan hệ cốt lõi: mục tiêu (ILO), hoạt động (TLA) và nhiệm vụ đánh giá (AT); cạnh nét đứt màu đỏ là liên kết không khớp mức Bloom; viền nét đứt màu cam đánh dấu mục tiêu có phép kiểm tra không đạt.

định trước khi chạy chính thức. Mỗi cặp (hệ, kịch bản) được sinh một lần. Khi một lần nộp bị từ chối, riêng hệ thống đề xuất viết lại sản phẩm đó một lần bằng giải mã có ràng buộc theo JSON Schema của vLLM [19], [20].

Ba LLM judge (mô hình LLM dùng để chấm) thuộc ba họ khác Qwen (GPT-6 Luna, Gemini 3.1 Flash-Lite, DeepSeek-V4.1-Flash) chấm mỗi đầu ra ở chế độ tắt suy luận; điểm cuối cùng là trung vị của ba điểm. GPT-6 Luna không nhận tham số temperature nên không tất định. Điểm ADDIE (0–100) đánh giá sản phẩm theo 33 tiêu chí con. Điểm Trajectory đánh giá quá trình gọi công cụ; LLM judge chỉ nhận 20 lời gọi đầu tiên [5], nên hệ có nhiều lời gọi như hệ thống đề xuất chịu bất lợi. Điểm Total bằng $0.7 \cdot \text{ADDIE} + 0.3 \cdot \text{Trajectory}$. Hệ thống đề xuất được so sánh với từng hệ còn lại bằng kiểm định Wilcoxon theo cặp kịch bản, hiệu chỉnh Holm trong từng điểm; hệ không tạo ra đầu ra chấm được ở một kịch bản nhận 0 điểm ở kịch bản đó.

B. Kết quả trên benchmark

Bảng II trình bày điểm LLM judge. Về điểm ADDIE, hệ thống đề xuất (59.0) không khác biệt có ý nghĩa với React-ADDIE (59.1; chênh lệch -0.1 điểm, KTC 95% [-2.3; 2.0], $p = 0.730$), Dick-Carey-Agent (58.0) và RPISD-Agent (58.4). Không khác biệt có ý nghĩa không đồng nghĩa với tương đương: khoảng cách giữa các hệ này nhỏ hơn nhiều so với độ lệch giữa các LLM judge trên cùng một đầu ra (8.7 điểm). Hệ thống đề xuất cao hơn ADDIE-Agent 10.5 điểm, Baseline 15.6 điểm và EduPlanner 28.9 điểm ($p < 0.001$ ở cả ba so sánh). Một phần chênh lệch với Baseline và EduPlanner đến từ các kịch bản không có đầu ra chấm được: Baseline thất bại

Bảng II
ĐIỂM LLM JUDGE TRÊN ISD-AGENT-BENCH (TRUNG BÌNH ± ĐỘ LỆCH CHUẨN QUA 30 KỊCH BẢN) VÀ TOKEN MỖI KỊCH BẢN

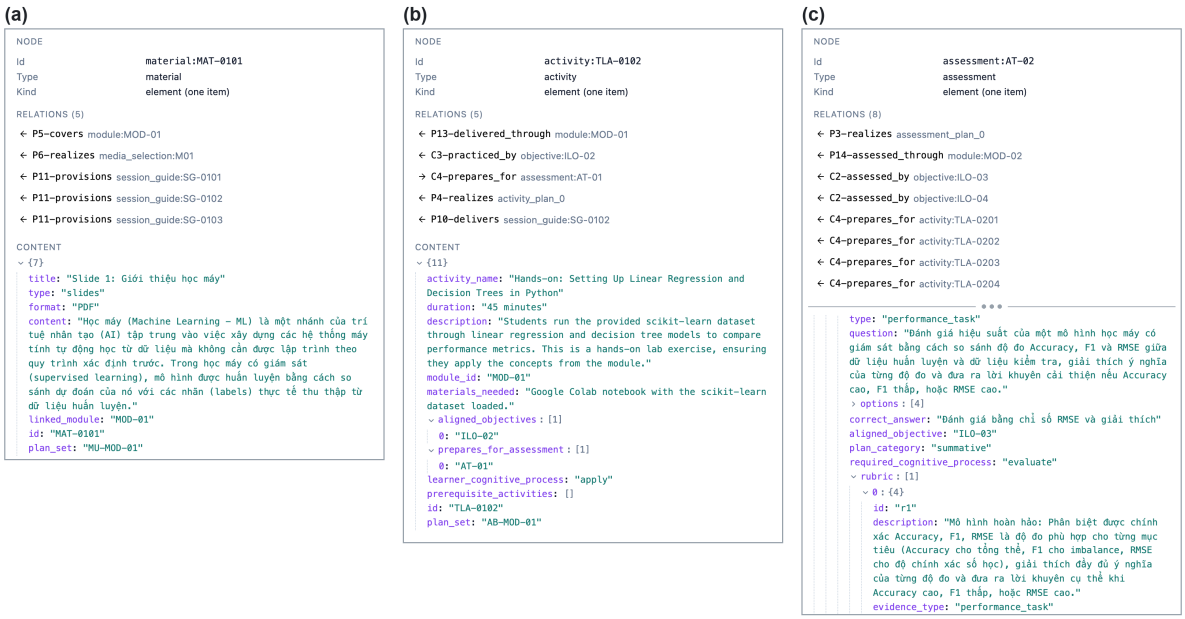
Hệ	ADDIE ↑	Trajectory ↑	Total ↑	Token (k) ↓
Hệ thống đề xuất	59.0 ± 2.8	69.7 ± 5.9	62.2 ± 2.3	282
EduPlanner	30.0 ± 19.8*	53.0 ± 32.7	36.9 ± 23.2*	70
Baseline	43.4 ± 17.7*	69.2 ± 19.0	51.1 ± 16.6*	10
React-ADDIE	$\mathbf{59.1 \pm 6.7}$	71.5 ± 9.5	$\mathbf{62.8 \pm 4.9}$	15
ADDIE-Agent	48.5 ± 3.6*	$\mathbf{81.5 \pm 3.0}$*	58.4 ± 2.7*	23
Dick-Carey-Agent	58.0 ± 3.0	72.6 ± 5.6	62.4 ± 3.1	26
RPISD-Agent	58.4 ± 2.5	69.9 ± 3.3	61.8 ± 1.9	61

* khác có ý nghĩa so với hệ thống đề xuất (Wilcoxon theo cặp, Holm trong từng điểm, $p < 0.05$).
Token trung bình trên các kịch bản có số liệu: EduPlanner 22, Baseline 28.
Kịch bản không có đầu ra chấm được, tính 0 điểm: EduPlanner 8, Baseline 2.

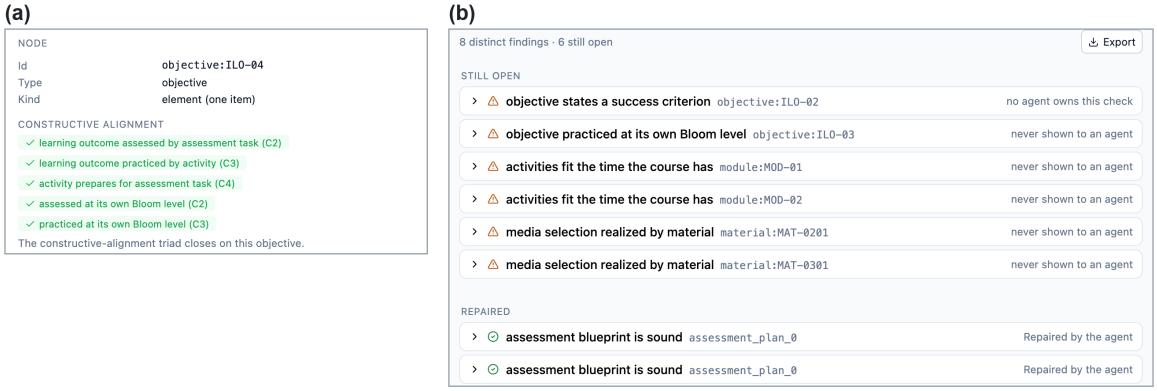
vì đầu ra chạm trần 8192 token, còn EduPlanner thất bại vì đầu ra không khớp lược đồ do chính nó quy định. Về điểm Trajectory, ADDIE-Agent đạt cao nhất (81.5) và cao hơn có ý nghĩa so với hệ thống đề xuất ($p < 0.001$). Về điểm Total, hệ thống đề xuất không khác biệt có ý nghĩa với React-ADDIE, Dick-Carey-Agent và RPISD-Agent. Hệ thống đề xuất dùng trung bình 282k token và 70 lời gọi mô hình mỗi kịch bản, so với 15k token và 5 lời gọi của React-ADDIE, gấp khoảng 18 lần về số token. Chúng tôi báo cáo chi phí này bên cạnh khả năng truy vết và kiểm toán, mà không diễn giải rằng hệ thống đề xuất hiệu quả hơn về mặt tính toán.

C. Nghiên cứu tình huống: học phần Nhập môn Học máy

Kiến trúc đề xuất được hiện thực thành một ứng dụng web. Giảng viên nhập bản mô tả yêu cầu thiết kế, hoặc mô tả học



Hình 3. Ba sản phẩm đầu ra trên giao diện, mỗi sản phẩm kèm các quan hệ của nó trong đồ thị. (a) Tài liệu MAT-0101 (slide), thuộc mô-đun MOD-01 và được dùng ở ba buổi học. (b) Hoạt động TLA-0102, luyện mục tiêu ILO-02 và chuẩn bị cho nhiệm vụ đánh giá AT-01. (c) Nhiệm vụ đánh giá tổng kết AT-02, đo hai mục tiêu và được bốn hoạt động chuẩn bị, cùng một câu hỏi của nó kèm tiêu chí rubric; dải chấm đánh dấu phần nội dung được lược bớt. Các quan hệ P ghi nguồn gốc và cách tổ chức theo mô-đun.



Hình 4. Đồ thị và sổ vi phạm trên giao diện. (a) Mục tiêu ILO-04 khép kín tam giác C_2–C_4 và các phép kiểm tra mức Bloom của nó. (b) Sổ vi phạm: các vi phạm còn mở và các vi phạm đã được sửa, kèm bên đã sửa.

phần bằng hội thoại, rồi khởi chạy hệ thống trên một SLM tự triển khai. Trong lúc hệ thống chạy, quy trình và đồ thị căn chỉnh được cập nhật dần trên màn hình. Sau khi chạy xong, giảng viên chọn từng nút để xem nội dung sản phẩm, các quan hệ của nó và các vi phạm còn mở.

Trong nghiên cứu tình huống, chúng tôi soạn bản mô tả yêu cầu cho học phần Nhập môn Học máy dành cho sinh viên Công nghệ Thông tin năm thứ ba: 4 tuần (30 giờ), học kết hợp, khoảng 40 sinh viên, thực hành Python trên Google Colab. Bản mô tả có bốn mục tiêu khóa học: quy trình học có giám sát, cài đặt hồi quy tuyến tính và cây quyết định, đánh giá mô hình, và xử lý quá khớp. Đầu ra yêu cầu bằng tiếng Việt. Hệ thống chạy với cùng cấu hình như Mục IV-A.

Hình 2a cho thấy quy trình thực hiện. Hệ thống sinh 5 mục tiêu, 3 mô-đun, 3 nhiệm vụ đánh giá với 8 câu hỏi, 10 hoạt động, 3 tài liệu và 8 kế hoạch buổi học, dùng 301k token qua 55 lời gọi mô hình. Mục tiêu khóa học thứ tư gồm hai hành vi (nhận diện quá khớp, áp dụng điều chuẩn) nên được tách thành ILO-04 và ILO-05; vì vậy bốn mục tiêu khóa học thành 5 mục tiêu.

Hình 3 trình bày ba loại sản phẩm đầu ra. Tài liệu MAT-0101 là slide giới thiệu học máy, viết bằng tiếng Việt. Hoạt động TLA-0102 là bài thực hành cài đặt hồi quy tuyến tính và cây quyết định trên Google Colab, kéo dài 45 phút. Câu hỏi tổng kết của AT-02 yêu cầu so sánh các độ đo giữa dữ liệu huấn luyện và dữ liệu kiểm tra, gắn với mục tiêu ILO-03 ở mức *evaluate* và có rubric mô tả từng mức đạt.

Bên cạnh nội dung, giảng viên cần biết bản thiết kế còn chỗ nào chưa khớp. Trên đồ thị (Hình 2b), độ phủ của C_2, C_3 và C_4 là 100%, 100% và 100%, và 5 trên 5 mục tiêu

khép kín tam giác (Hình 4a). Tuy vậy, sổ vi phạm (Mục III-D, Hình 4b) vẫn ghi 8 vi phạm: 2 đã được tác tử sửa, 6 còn mở. Các vi phạm còn mở chưa được đưa tới tác tử nào, vì được phát hiện sau khi tác tử phụ trách đã kết thúc hoặc thuộc phép kiểm tra chưa có tác tử phụ trách. Chúng chỉ cho giảng viên những chỗ cần xem lại: ILO-02 chưa nêu tiêu chí đạt, hoạt động của ILO-03 thấp hơn mức Bloom của mục tiêu (cạnh nét đứt màu đỏ ở Hình 2b), tổng thời lượng hoạt động của hai mô-đun vượt thời lượng của mô-đun, và hai tài liệu khai loại mô phỏng nhưng thực chất là kịch bản video. Bản thiết kế còn những lỗi nằm ngoài phạm vi của các phép kiểm tra. ILO-01 và ILO-02 chỉ được đo bởi một nhiệm vụ chẩn đoán có trọng số 0%, nên tam giác khép kín về cấu trúc nhưng chưa có đánh giá tổng kết. Mô-đun thứ hai có các kế hoạch buổi học chồng lấn nhau, và trường mục tiêu của kế hoạch buổi học trỏ tới id không tồn tại mà không bị phát hiện, vì không quan hệ nào đọc trường đó. Mục tiêu, câu hỏi, rubric và tài liệu được viết bằng tiếng Việt, nhưng kế hoạch đánh giá, các sản phẩm của pha triển khai và các hoạt động của mô-đun đầu tiên được viết bằng tiếng Anh (Hình 3b); một phần do lớp điều phối sinh từ mẫu cố định. Một mục tiêu gọi kiểm chứng chéo là kỹ thuật điều chuẩn. Độ phủ quan hệ và tam giác khép kín hỗ trợ truy vết cấu trúc nhưng không thay thế thẩm định chuyên môn của giảng viên. Các lỗi này cho thấy bản thiết kế do SLM sinh ra vẫn cần giảng viên rà soát trước khi dùng.

D. Thảo luận

(1) Đầu ra truy vết được. Mỗi quan hệ mục tiêu–hoạt động–đánh giá là một cạnh trên giao diện, nên giảng viên thấy mục tiêu nào chưa được đánh giá hay chưa có hoạt động luyện tập mà không phải đọc lại toàn bộ bản thiết kế. Các phép kiểm tra chỉ là kiểm tra cấu trúc, không phát hiện lỗi chuyên môn hay lỗi ngôn ngữ.

(2) Nền tảng cho cải tiến cục bộ. Với vi phạm còn mở của ILO-03, đồ thị chỉ ra bốn hoạt động là ứng viên cần viết lại và 8 sản phẩm liền kề cần rà soát cùng: nhiệm vụ đánh giá, mô-đun, mục tiêu và kế hoạch buổi học liên quan; 23 trên 35 sản phẩm thiết kế nằm ngoài lân cận một cạnh này. Đây là phạm vi đọc trực tiếp từ đồ thị, chưa phải phân tích lan truyền.

Hạn chế. Thực nghiệm chỉ dùng một mô hình (Qwen3.5-2B), 30 kịch bản và một lần sinh cho mỗi cặp. LLM judge có những thiên lệch đã được ghi nhận [21]; độ lệch chuẩn giữa ba LLM judge trên cùng một đầu ra trung bình 8.7 điểm ADDIE. Ba LLM judge này thay cho cặp LLM judge mặc định trong mã nguồn của benchmark (GPT-4o-mini và Gemini 2.5 Flash-Lite), và thứ hạng giữa hệ thống đề xuất và React-ADDIE thay đổi theo từng LLM judge. Hệ thống đề xuất được hoàn thiện trên chính mô hình này và có thêm bước giải mã có ràng buộc, trong khi các tác tử đối chứng chạy nguyên phương pháp; phép so sánh vì vậy đánh giá cả gói hệ thống, không tách riêng đóng góp của kiến trúc. Nghiên cứu tình huống chỉ gồm một học phần, một lần chạy, chưa được đưa vào giảng dạy và chưa có đánh giá của chuyên gia hay của người học. Với đầu ra tiếng Việt, SLM 2B còn trộn ngôn ngữ và mắc lỗi chuyên môn, và các mẫu văn bản của lớp điều phối chưa được bản địa hóa.

V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Nghiên cứu này đã xây dựng và đánh giá một hệ thống đa tác tử hỗ trợ thiết kế giảng dạy trên SLM tự triển khai, trong đó các tác tử của sáu công đoạn dùng chung một đồ thị căn chỉnh. Điểm ADDIE của hệ thống không khác biệt có ý nghĩa với tác tử mạnh nhất của benchmark, với chi phí token cao hơn khoảng 18 lần, và sổ vi phạm chỉ ra cho giảng viên những chỗ cần xem lại. Bản thiết kế vẫn cần giảng viên rà soát trước khi dùng. Hướng phát triển gồm vòng chỉnh sửa có giảng viên tham gia và sinh lại cục bộ theo đồ thị, đánh giá trên nhiều cỡ mô hình, và lấy nhận xét của chuyên gia.

TÀI LIỆU THAM KHẢO

[1] R. M. Branch, *Instructional Design: The ADDIE Approach*. New York, NY: Springer, 2009.

[2] X. Zeng, Y. Zheng, S. Huang *et al.*, "COMA: A collaborative multi-role agent framework for automated lesson plan generation," in *Proc. ACM Web Conf. (WWW)*, 2026, pp. 9745–9756.

[3] X. Zhang, C. Zhang, J. Sun *et al.*, "EduPlanner: LLM-based multiagent systems for customized and intelligent instructional design," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 416–427, 2025.

[4] H. Yao, W. Xu, J. Turnau *et al.*, "Instructional agents: Reducing teaching faculty workload through multi-agent instructional design," in *Proc. EACL (Long Papers)*, 2026, pp. 4087–4109.

[5] Y. Jeon, S. Kim, H. Son *et al.*, "ISD-Agent-Bench: A comprehensive benchmark for evaluating LLM-based instructional design agents," arXiv preprint arXiv:2602.10620, 2026, mã nguồn: https://github.com/codingchild2424/isd-agent-benchmark.

[6] J. Robert and M. McCormack, "2025 EDUCAUSE AI landscape study: Into the digital AI divide," EDUCAUSE, Tech. Rep., 2025.

[7] L. Yan, L. Sha, L. Zhao *et al.*, "Practical and ethical challenges of large language models in education: A systematic scoping review," *British Journal of Educational Technology*, vol. 55, no. 1, pp. 90–112, 2024.

[8] C. K. Y. Chan, W. Dai, K. Cao *et al.*, "Bigger isn't always better: Choosing the right size large language model for locally hosted school settings," *Applied Sciences*, vol. 16, no. 11, p. 5268, 2026.

[9] K. Katharakis, S. Rossi, and R. R. Mukkamala, "Small language models for curriculum-based guidance," in *Proc. 59th Hawaii Int. Conf. System Sciences (HICSS)*, 2026.

[10] W. Shen, C. Li, H. Chen *et al.*, "Small LLMs are weak tool learners: A multi-LLM agent," in *Proc. EMNLP*, 2024, pp. 16 658–16 680.

[11] J. Biggs, "Enhancing teaching through constructive alignment," *Higher Education*, vol. 32, no. 3, pp. 347–364, 1996.

[12] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[13] S. Hong, M. Zhuge, J. Chen *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.

[14] T. Guo, X. Chen, Y. Wang *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. IJCAI*, 2024, pp. 8048–8057.

[15] E. Pereira, S. Nsair, L. R. Pereira *et al.*, "Constructive alignment in a graduate-level project management course: an innovative framework using large language models," *International Journal of Educational Technology in Higher Education*, vol. 21, no. 1, p. 25, 2024.

[16] L. W. Anderson and D. R. Krathwohl, Eds., *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York, NY: Longman, 2001.

[17] G. Wiggins and J. McTighe, *Understanding by Design*, 2nd ed. Alexandria, VA: Association for Supervision and Curriculum Development (ASCD), 2005.

[18] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023, pp. 611–626.

[19] vLLM Team, "Structured outputs," https://docs.vllm.ai/en/latest/features/structured_outputs/, 2026, truy cập 14/10/2026.

[20] Y. Dong, C. F. Ruan, Y. Cai *et al.*, "XGrammar: Flexible and efficient structured generation engine for large language models," in *Proc. MLSys*, 2025.

[21] J. Ye, Y. Wang, Y. Huang *et al.*, "Justice or prejudice? quantifying biases in LLM-as-a-judge," in *Proc. ICLR*, 2025.